Logistic Regression — Complete Theory Guide

Nov 7, 2026 · @Razeen

1. Introduction to Logistic Regression

Logistic Regression is a **supervised machine learning algorithm** used for **classification** — predicting which category an input belongs to. Despite the word "Regression" in its name, it is used for classification, not for predicting continuous numbers.

Key Terms

Term	Meaning
Supervised Learning	Model learns from labelled data (input + correct answer)
Classification	Predicting a category (e.g., Pass/Fail, Spam/Not Spam)
Binary Classification	Only 2 possible output classes (0 or 1)
Target / Label (y)	The column we want to predict
Features / Input (X)	Columns used to make the prediction
Probability	A number between 0 and 1 expressing likelihood

Why Logistic Regression?

Logistic Regression is the go-to starting point for binary classification because:

- It is fast and interpretable
- It outputs a **probability** (0 to 1), not just a class label
- It works well when the relationship between features and the log-odds is approximately linear
- It is the backbone of many advanced models (neural networks, GLMs)

Real-World Examples

- Will this patient have diabetes? (Yes/No)
- Will this email be spam? (Spam/Not Spam)

- Will this customer churn? (Churn/Stay)
- Will this loan applicant default? (Default/No Default)

2. The Logistic (Sigmoid) Function

The **sigmoid function** (also called the logistic function) is the mathematical heart of Logistic Regression. It maps any real number to a value between 0 and 1.

Formula

```
\sigma(z) = \frac{1}{1 + e^{-z}}
```

Where:

- **z** = any real number (the raw score from the linear equation)
- **e** = Euler's number ≈ 2.718
- **σ(z)** = output probability between 0 and 1

Key Properties

Input (z)	Output σ(z)	Interpretation
Very large positive	≈ 1.0	Almost certain it is class 1
0	0.5	50/50, completely uncertain
Very large negative	≈ 0.0	Almost certain it is class 0

Decision Rule

By default, we use a **threshold of 0.5**:

- If σ(z) ⩾ 0.5 → Predict class **1**
- If σ(z) < 0.5 → Predict class **0**

This threshold can be changed depending on the use case (e.g., lowering it to 0.3 for cancer detection so we miss fewer positive cases).

Why the Sigmoid?

Before the sigmoid, we only had a raw score (z) that could be any number. The sigmoid squishes this into (0, 1), making it interpretable as a probability. This is why logistic regression outputs probabilities, not just class labels.

3. From Linear Regression to Logistic Regression

The Problem with Linear Regression for Classification

Linear Regression predicts a continuous value: house price, temperature, salary. Its output is unbounded — it can be negative or greater than 1. For classification, we need outputs between 0 and 1.

Linear Regression equation:

```
\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + \ldots + w_n x_n
```

Problem: `y_hat` can be -5 or 200, which makes no sense as a probability.

The Logistic Regression Fix

We wrap the linear equation inside the sigmoid function:

```
P(y = 1 \mid X) = \sigma(w_0 + w_1 x_1 + \ldots + w_n x_n) = \frac{1}{1 + e^{-(w_0 + w_1 x_1 + \ldots + w_n x_n)}}
```

Now the output is always between 0 and 1 — a valid probability.

Comparison Table

Property	Linear Regression	Logistic Regression
Task	Predicts a number	Predicts a category
Output range	-∞ to +∞	0 to 1 (probability)
Loss function	Mean Squared Error (MSE)	Log Loss (Binary Cross-Entropy)
Activation	None (identity)	Sigmoid
Use case	House price, salary	Spam, churn, disease

Intuition with an Example

Suppose a student studies for 6 hours. The linear equation gives z = 2.4. The sigmoid gives σ(2.4) = 0.917 — a 91.7% chance of passing. We predict **Pass (1)**.

4. Linear to Logistic — The Math

Step 1: Start with the Linear Equation

$$z = w_0 + w_1 x_1 + w_2 x_2 + \ldots + w_n x_n$$

Here w_0 is the **intercept (bias)** and $w_1 \ldots w_n$ are the **weights (coefficients)**.

Step 2: Convert to Probability via Sigmoid

$$P = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Step 3: The Log-Odds (Logit) Form

If we rearrange the sigmoid equation, we get the **logit**:

$$\ln\left(\frac{P}{1 - P}\right) = w_0 + w_1 x_1 + \ldots + w_n x_n$$

- **Odds** = P / (1 − P) = ratio of probability of success to failure
- **Log-Odds (Logit)** = natural log of the odds
- The logit is a **linear function** of the features

This is why logistic regression is called a **Generalized Linear Model (GLM)** — it is linear on the log-odds scale.

Step 4: Coefficient Interpretation

For a feature x_1 with coefficient w_1:

- Each unit increase in x_1 increases the log-odds by w_1
- Equivalently, the odds are multiplied by $\mathbf{e^{w_1}}$
- Positive w_1 → higher probability of class 1
- Negative w_1 → lower probability of class 1

Example: Exam Pass

Suppose the model learns:

$$z = -4.5 + 1.2 \times \text{Hours_Studied}$$

For a student who studied 5 hours:

$$z = -4.5 + 1.2 \times 5 = 1.5$$

$$P(\text{Pass}) = \frac{1}{1 + e^{-1.5}} = \frac{1}{1 + 0.223} \approx 0.818$$

The student has an 81.8% probability of passing.

5. Maximum Likelihood Estimation (MLE)

How Does the Model Learn?

Logistic regression finds the best weights (w_0, w_1, …) by **Maximum Likelihood Estimation (MLE)**.

Likelihood = the probability that our model would have produced the actual observed data, given a set of weights.

MLE finds the weights that **maximise** this likelihood — i.e., make the observed data as probable as possible.

The Log-Likelihood

Working with probabilities involves multiplying many small numbers, which can underflow. Instead we take the **log** (which turns products into sums) and maximise the **log-likelihood**:

$$\ell(\mathbf{w}) = \sum_{i=1}^{N} \left[y_i \ln(\hat{p}_i) + (1 - y_i) \ln(1 - \hat{p}_i) \right]$$

The Loss Function: Binary Cross-Entropy (Log Loss)

In practice we **minimise** the **negative** log-likelihood, which is called **Log Loss** or **Binary Cross-Entropy**:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \left[y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i) \right]$$

Where:

- N = number of training samples
- y_i = actual label (0 or 1)
- p_hat_i = predicted probability for sample i

How Log Loss Penalises Wrong Predictions

Actual y	Predicted p	Log Loss contribution	Explanation
1	0.95	≈0.05	Correct & confident — low penalty
1	0.50	0.69	Correct but unsure — moderate penalty
1	0.05	≈3.0	Completely wrong — huge penalty
0	0.05	≈0.05	Correct & confident — low penalty

Optimisation

There is no closed-form solution (unlike linear regression). We use iterative algorithms:

- **Gradient Descent** — moves weights in the direction that reduces loss
- **L-BFGS** — a quasi-Newton method, default solver in sklearn; faster for small data
- **Newton-Raphson** — uses second derivatives; converges faster but costs more per step

Scikit-learn handles all of this automatically; you just call `.fit(X, y)`.

6. EDA and Model Training

EDA (Exploratory Data Analysis)

Before training any model, always explore your data. EDA helps you:

- Understand the distribution of each feature
- Detect missing values and outliers
- Identify correlations between features and the target
- Check class balance (imbalance → accuracy is misleading)

Key EDA steps for classification:

1. `df.shape` — how many rows and columns?
2. `df.dtypes` — what is the data type of each column?
3. `df.describe()` — summary statistics (mean, std, min, max)
4. `df.isnull().sum()` — count of missing values per column
5. `df['target'].value_counts()` — is the dataset balanced?
6. Visualise distributions with histograms and bar charts
7. Correlation heatmap for numeric features

Preprocessing Checklist

Step	Why	How
Handle missing values	Most models cannot handle NaN	`fillna(median)` or `fillna(mode)`
Encode categorical columns	Models need numbers	`LabelEncoder` or `pd.get_dummies`
Feature scaling	Keeps coefficients comparable	`StandardScaler`
Train-test split	Evaluate on unseen data	`train_test_split(test_size=0.2)`
Stratify the split	Preserve class balance	`stratify=y`

Training the Model

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(solver='lbfgs', max_iter=1000, C=1.0,
random_state=42)
model.fit(X_train_scaled, y_train)
```

Key Parameters:

Parameter	Meaning	Typical value
solver	Optimisation algorithm	'lbfgs' (default, good for small data)
max_iter	Maximum iterations	1000 (increase if model gives a warning)
C	Inverse of regularisation strength	1.0 (smaller = more regularisation)
random_state	Reproducibility seed	Any integer (e.g., 42)

StandardScaler — Why Scale?

Logistic regression uses a gradient-based optimiser. If feature A ranges from 0–1 and feature B ranges from 0–100,000, the gradient steps for B dominate and training becomes slow or unstable. Scaling makes all features comparable:

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma}$$

Always fit the scaler on **training data only**, then apply it to test data.

7. Confusion Matrix and Accuracy

What is a Confusion Matrix?

A **confusion matrix** is a 2×2 table (for binary classification) that shows the counts of correct and incorrect predictions.

```
                 Predicted Negative (0)   Predicted Positive (1)
Actual Negative (0)       TN                       FP
Actual Positive (1)       FN                       TP
```

Term	Full Name	Meaning
TP	True Positive	Predicted 1, actually 1 ✅
TN	True Negative	Predicted 0, actually 0 ✅
FP	False Positive	Predicted 1, actually 0 ❌ (Type I Error)
FN	False Negative	Predicted 0, actually 1 ❌ (Type II Error)

Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Accuracy is the simplest metric: the fraction of predictions that were correct.

When accuracy misleads: If 95% of emails are not spam and the model always predicts "not spam", it gets 95% accuracy — but it never detects any spam at all. This is the **accuracy paradox** on imbalanced datasets.

Python Code (Both Styles)

Older style — seaborn heatmap:

```
import seaborn as sns
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Neg', 'Pos'], yticklabels=['Neg', 'Pos'])
```

Modern style (recommended) — ConfusionMatrixDisplay:

```
from sklearn.metrics import ConfusionMatrixDisplay
ConfusionMatrixDisplay.from_estimator(model, X_test, y_test, cmap='Blues')
```

`ConfusionMatrixDisplay` is cleaner: it handles labels and colours automatically and is the current sklearn standard.

8. Precision, Recall, and F1 Score

Precision

Of all the samples the model **predicted as positive**, how many were **actually positive**?

```
\text{Precision} = \frac{TP}{TP + FP}
```

High precision = fewer false alarms. Use when **False Positives are costly**. Example: Spam detection — if a legitimate email is marked spam, the user misses an important message.

Recall (Sensitivity / True Positive Rate)

Of all the samples that were **actually positive**, how many did the model **correctly identify**?

```
\text{Recall} = \frac{TP}{TP + FN}
```

High recall = fewer missed cases. Use when **False Negatives are costly**. Example: Cancer screening — missing a cancer diagnosis is far worse than a false alarm.

F1 Score

The **harmonic mean** of Precision and Recall. Balances both metrics into a single number.

```
F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}
```

F1 = 1.0 is perfect. F1 is better than accuracy for imbalanced datasets.

Precision vs Recall Trade-off

Changing the classification threshold shifts the balance:

- **Higher threshold** (e.g., 0.7) → more confident positives → higher Precision, lower Recall
- **Lower threshold** (e.g., 0.3) → catch more positives → higher Recall, lower Precision

Which Metric to Use?

Situation	Preferred Metric	Reason
Spam filter	Precision	Don't block real emails
Cancer detection	Recall	Don't miss cancer cases
Fraud detection	F1	Balance both
Class imbalance	F1 or Balanced Accuracy	Accuracy is misleading
Balanced classes	Accuracy	Simple and intuitive

Classification Report

Scikit-learn's `classification_report` prints all metrics at once:

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred, target_names=['No', 'Yes']))
```

Output columns: precision, recall, f1-score, support (number of actual samples per class), plus macro avg and weighted avg.

9. ROC Curves

What is a ROC Curve?

The **Receiver Operating Characteristic (ROC) curve** shows the performance of a classification model **at every possible decision threshold**. It plots:

- **X-axis**: False Positive Rate (FPR) = FP / (FP + TN)
- **Y-axis**: True Positive Rate (TPR / Recall) = TP / (TP + FN)

```
\text{TPR} = \frac{TP}{TP + FN} \qquad \text{FPR} = \frac{FP}{FP + TN}
```

How to Read the ROC Curve

- The **diagonal line** (from bottom-left to top-right) = a random classifier (AUC = 0.5)
- A curve that **hugs the top-left corner** = a better model
- A perfect model = a point at (0, 1) = AUC of 1.0

AUC — Area Under the Curve

The **AUC (Area Under the ROC Curve)** summarises the entire ROC curve in one number:

AUC Value	Interpretation
1.0	Perfect model
0.9 – 1.0	Excellent
0.8 – 0.9	Good
0.7 – 0.8	Fair
0.6 – 0.7	Poor
0.5	No better than random
< 0.5	Worse than random (labels may be flipped)

Python Code (Both Styles)

Older style — manual using roc_curve:

```
from sklearn.metrics import roc_curve, roc_auc_score
import matplotlib.pyplot as plt

fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob)
auc = roc_auc_score(y_test, y_pred_prob)

plt.plot(fpr, tpr, label=f'AUC = {auc:.2f}')
plt.plot([0, 1], [0, 1], 'k--')  # random baseline
plt.xlabel('FPR'); plt.ylabel('TPR')
plt.legend()
plt.show()
```

Modern style (recommended) — RocCurveDisplay:

```
from sklearn.metrics import RocCurveDisplay
RocCurveDisplay.from_estimator(model, X_test, y_test)
plt.plot([0, 1], [0, 1], 'k--')  # random baseline
plt.show()
```

`RocCurveDisplay.from_estimator` automatically computes probabilities, draws the curve, and labels it with the AUC.

Key Insight

The ROC curve is **threshold-independent** — it evaluates the model across all possible thresholds. AUC tells you how well the model **separates** the two classes, regardless of which threshold you ultimately choose.

10. Logistic Regression Performance Evaluation

This section brings all metrics together and explains how to interpret a model holistically.

Complete Metric Reference

Metric	Formula	Range	Best Value	Use When
Accuracy	(TP+TN) / Total	0 to 1	1.0	Balanced classes
Precision	TP / (TP+FP)	0 to 1	1.0	FP is costly
Recall	TP / (TP+FN)	0 to 1	1.0	FN is costly
F1 Score	2 × (P×R)/(P+R)	0 to 1	1.0	Imbalanced data
Specificity	TN / (TN+FP)	0 to 1	1.0	Correctly identify negatives
AUC-ROC	Area under ROC	0 to 1	1.0	Threshold-independent
Log Loss	$-(1/N)\Sigma[y \log p]$	0 to ∞	0	Probabilistic quality
Balanced Accuracy	(Recall + Specificity) / 2	0 to 1	1.0	Imbalanced classes

Evaluation Workflow

1. Train the model on `X_train`
2. Predict on `X_test` — get both class labels (`predict`) and probabilities (`predict_proba`)
3. Compute the confusion matrix — understand TP, TN, FP, FN
4. Compute classification report — precision, recall, F1 per class
5. Compute AUC-ROC — overall discriminative power
6. Plot confusion matrix + ROC curve visually

Regularisation and Underfitting / Overfitting

Issue	Symptom	Fix
Overfitting	High train accuracy, low test accuracy	Increase regularisation (smaller C)
Underfitting	Low train AND test accuracy	Decrease regularisation (larger C) or add features

C in sklearn is the inverse of regularisation strength:

- Small C (e.g., 0.01) = strong regularisation = simpler model = may underfit
- Large C (e.g., 100) = weak regularisation = complex model = may overfit
- Default C = 1.0 is usually a good starting point

11. Multiclass Classification with Logistic Regression

Beyond Binary: 3 or More Classes

Logistic regression can be extended to predict **more than 2 classes**. The two main strategies are:

Strategy 1: One-vs-Rest (OvR)

Train one binary classifier per class:

- Classifier 1: Class A vs (B, C)
- Classifier 2: Class B vs (A, C)
- Classifier 3: Class C vs (A, B)

For a new sample, run all classifiers and pick the class with the **highest probability**.

In sklearn: `multi_class='ovr'` (or the default for many solvers).

Strategy 2: Multinomial (Softmax)

Train **one model** that directly outputs a probability for each class using the **Softmax function** (a generalisation of sigmoid):

```
P(y = k \mid X) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}
```

Where K = number of classes and z_k = raw score for class k.

Properties:

- All class probabilities sum to 1
- The class with the highest probability is predicted
- More principled than OvR for truly multi-class problems

In sklearn: `multi_class='multinomial', solver='lbfgs'`

Python Code

```
from sklearn.linear_model import LogisticRegression

# Multinomial logistic regression
model = LogisticRegression(multi_class='multinomial', solver='lbfgs',
max_iter=1000)
model.fit(X_train_scaled, y_train)

# Predict probabilities for all classes
probs = model.predict_proba(X_test_scaled)
print('Classes:', model.classes_)
print('Probabilities shape:', probs.shape)  # (n_samples, n_classes)
```

Evaluation for Multiclass

- Use `classification_report` — prints precision, recall, F1 per class
- Use `ConfusionMatrixDisplay.from_estimator` — shows a K×K matrix
- **Macro avg** — mean of per-class metrics; treats all classes equally
- **Weighted avg** — mean weighted by class size; better for imbalanced classes

Averaging	Formula	When to Use
Macro	Mean of class metrics	All classes equally important
Weighted	Class-size-weighted mean	Imbalanced datasets
Micro	Global TP/FP/FN pooled	Each sample equally important

12. Quick Reference Cheat Sheet

All Key Formulas

Concept	Formula
Sigmoid	σ(z) = 1 / (1 + e^-z)

Concept	Formula
Linear score	$z = w_0 + w_1x_1 + \ldots + w_nx_n$
Log-Odds	$\ln(P / 1-P) = z$
Log Loss	$-(1/N)\ \Sigma\ [y \log(\hat{p}) + (1-y) \log(1-\hat{p})]$
Accuracy	(TP + TN) / (TP + TN + FP + FN)
Precision	TP / (TP + FP)
Recall	TP / (TP + FN)
Specificity	TN / (TN + FP)
F1 Score	2 × (P × R) / (P + R)
TPR	TP / (TP + FN)
FPR	FP / (FP + TN)
Softmax	P(y=k) = e^(z_k) / Σ e^(z_j)
Feature scaling	(x − μ) / σ

sklearn One-Liners

```
# Import
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score, confusion_matrix, ConfusionMatrixDisplay,
    classification_report, roc_auc_score, RocCurveDisplay
)

# Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
stratify=y, random_state=42)

# Scale
sc = StandardScaler()
X_train_sc = sc.fit_transform(X_train)
X_test_sc  = sc.transform(X_test)

# Train
model = LogisticRegression(C=1.0, max_iter=1000, random_state=42)
model.fit(X_train_sc, y_train)
```

```
# Predict
y_pred  = model.predict(X_test_sc)
y_proba = model.predict_proba(X_test_sc)[:, 1]

# Evaluate
print(accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
ConfusionMatrixDisplay.from_estimator(model, X_test_sc, y_test)
RocCurveDisplay.from_estimator(model, X_test_sc, y_test)
```

Common Mistakes to Avoid

Mistake	Correct Approach
Scaling test data with test stats	Always `fit` scaler on train only, then `transform` test
Using accuracy on imbalanced data	Use F1, Recall, AUC instead
Not checking class balance	Always `y.value_counts()` before training
Ignoring convergence warnings	Increase `max_iter` or change `solver`
Using LinearRegression for classification	Use LogisticRegression
Applying `predict_proba` without sigmoid	sklearn handles this automatically

Glossary of All Keywords

Term	Meaning
Classification	Predicting a category, not a number
Sigmoid	S-shaped function mapping any value to (0, 1)
Log-Odds / Logit	Natural log of odds ratio; linear in features
MLE	Method to find weights that make data most likely
Log Loss	Loss function minimised during training
Threshold	Cutoff probability for class assignment (default 0.5)

Term	Meaning
TP / TN / FP / FN	Four cells of the confusion matrix
Precision	Accuracy among predicted positives
Recall	Coverage of actual positives
F1 Score	Harmonic mean of Precision and Recall
ROC Curve	TPR vs FPR at every threshold
AUC	Single number summarising the ROC curve
Overfitting	Model memorises training data, fails on new data
Regularisation	Penalty on large weights to prevent overfitting
C parameter	Inverse regularisation strength in sklearn
Softmax	Multi-class generalisation of sigmoid
OvR	One-vs-Rest: K binary classifiers for K classes
StandardScaler	Transform features to mean=0, std=1
Stratify	Preserve class balance in train/test split